

Cinemática de robots terrestres

Juan Francisco Rascón Crespo

2 de marzo de 2026

1. Restricciones de las ruedas

Completar

2. Base con tres ruedas activas orientables (three swerve drive)

La base con tres ruedas activas orientables (three swerve drive) es un sistema de locomoción omnidireccional. Permite que un robot terrestre se mueva en cualquier dirección sin tener que cambiar antes la orientación del chasis. Cada rueda se orienta de forma independiente, por lo que el robot gana maniobrabilidad y flexibilidad.

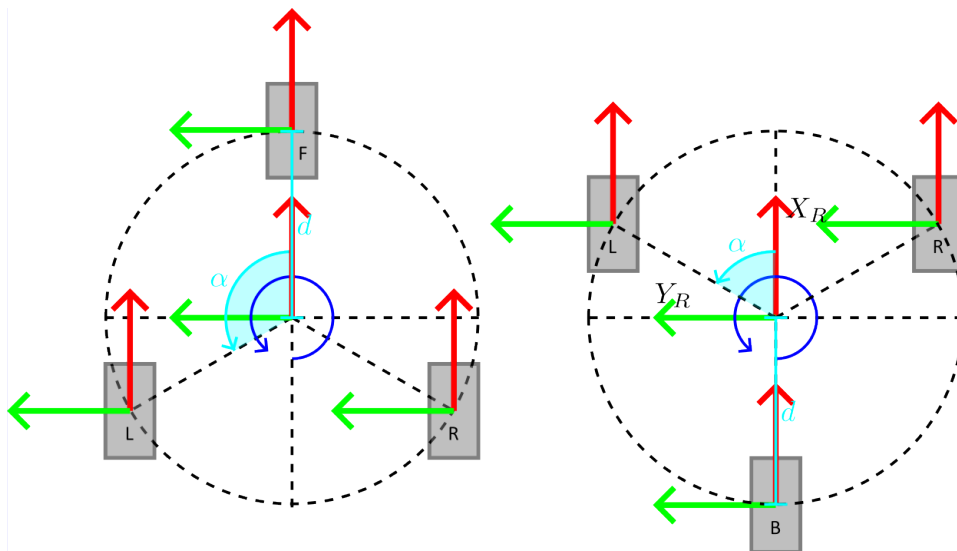


Figura 1: Configuraciones comunes de las ruedas en un sistema three swerve drive (izquierda: configuración Y-invertida, derecha: configuración Y)

La disposición típica de las ruedas es triangular. Las dos configuraciones más comunes son:

- Configuración Y-invertida: Una rueda se coloca a distancia d del centro sobre el eje X_R positivo. Las otras dos se colocan también a distancia d , en los ángulos α° y $-\alpha^\circ$ respecto del eje X_R positivo, con $\alpha \in [90^\circ, 180^\circ)$.
- Configuración Y: Una rueda se coloca a distancia d del centro sobre el eje X_R negativo. Las otras dos se colocan también a distancia d , en los ángulos α° y $-\alpha^\circ$ respecto del eje X_R positivo, con $\alpha \in (0^\circ, 90^\circ]$.

La figura 1 muestra ambas configuraciones.

A continuación se presentan las expresiones de la cinemática inversa y directa de un robot con tres ruedas activas para las configuraciones anteriores. También puede considerarse una formulación más general, con distancias al centro distintas o con ruedas en ángulos no simétricos. Sin embargo, estas configuraciones son menos frecuentes y, en este contexto, solo complican las expresiones sin aportar ventajas prácticas, ya que, en la industria, se emplean principalmente las configuraciones descritas anteriormente.

Para la configuración Y-invertida, se usará $i \in \{F, L, R\}$ para identificar cada rueda. Para la configuración Y, se usará $i \in \{B, L, R\}$.

La velocidad lineal de la base se denota como:

$$\mathbf{V} = \begin{bmatrix} V_x \\ V_y \\ 0 \end{bmatrix} \quad (1)$$

y se expresa en m/s.

La velocidad angular de la base se denota como:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega_z \end{bmatrix} \quad (2)$$

La velocidad lineal de cada rueda i se expresa como:

$$\begin{aligned} \mathbf{V}_i &= \mathbf{V} + \boldsymbol{\omega} \times \mathbf{d}_i \\ &= \mathbf{V} + [\boldsymbol{\omega}]_{\times} \mathbf{d}_i \end{aligned} \quad (3)$$

donde $\mathbf{d}_i$ es el vector de posición de la rueda i respecto al centro de la base.

El producto vectorial entre $\boldsymbol{\omega}$ y $\mathbf{d}_i$ puede escribirse como un producto matricial mediante el operador antisimétrico $[\cdot]_{\times}$: $\boldsymbol{\omega} \times \mathbf{d}_i = [\boldsymbol{\omega}]_{\times} \mathbf{d}_i$, y $[\boldsymbol{\omega}]_{\times}$ se

define como:

$$[\boldsymbol{\omega}]_{\times} \triangleq \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

Por lo tanto, considerando $\boldsymbol{\omega} = [0 \ 0 \ \omega_z]^T$ y $\mathbf{d}_i = [d \cos \alpha_i \ d \sin \alpha_i \ 0]^T$:

$$\mathbf{V}_i = \mathbf{V} + \begin{bmatrix} -\omega_z d \sin \alpha_i \\ \omega_z d \cos \alpha_i \\ 0 \end{bmatrix} = \begin{bmatrix} V_x - \omega_z d \sin \alpha_i \\ V_y + \omega_z d \cos \alpha_i \\ 0 \end{bmatrix} \quad (4)$$

La magnitud de la velocidad lineal de cada rueda i se expresa como:

$$\|\mathbf{V}_i\| = \sqrt{(V_x - \omega_z d \sin \alpha_i)^2 + (V_y + \omega_z d \cos \alpha_i)^2} = \omega_i r \quad (5)$$

donde r es el radio de la rueda y ω_i es la velocidad angular de la rueda i .

Obviamente, todas las ruedas tienen el mismo radio r .

2.1. Cinemática inversa

Se denomina cinemática inversa al proceso de calcular las velocidades de las ruedas ω_i , y su orientación θ_i , a partir de la velocidad lineal $\mathbf{V}$ y la velocidad angular $\boldsymbol{\omega}$ de la base.

Se calcula la orientación de cada rueda i , θ_i , a partir de su velocidad lineal $\mathbf{V}_i$:

$$\theta_i = \arctan 2(V_{iy}, V_{ix}) = \arctan 2(V_y + \omega_z d \cos \alpha_i, V_x - \omega_z d \sin \alpha_i) \quad (6)$$

A continuación, se obtiene la velocidad angular de cada rueda i , ω_i , a partir de su velocidad lineal, $\mathbf{V}_i$, y su radio, r :

$$\omega_i = \frac{\|\mathbf{V}_i\|}{r} = \frac{\sqrt{(V_x - \omega_z d \sin \alpha_i)^2 + (V_y + \omega_z d \cos \alpha_i)^2}}{r} \quad (7)$$

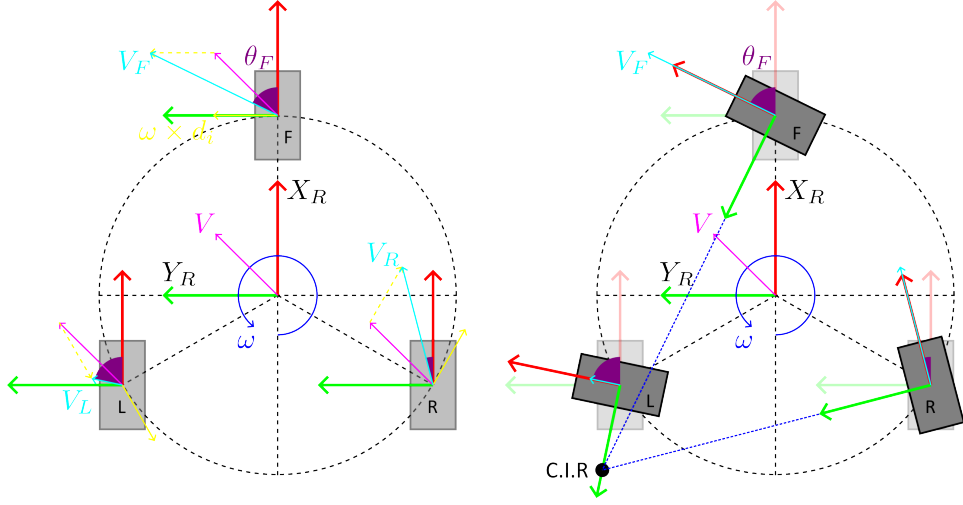


Figura 2: Cinemática inversa de un robot con configuración Y-invertida.

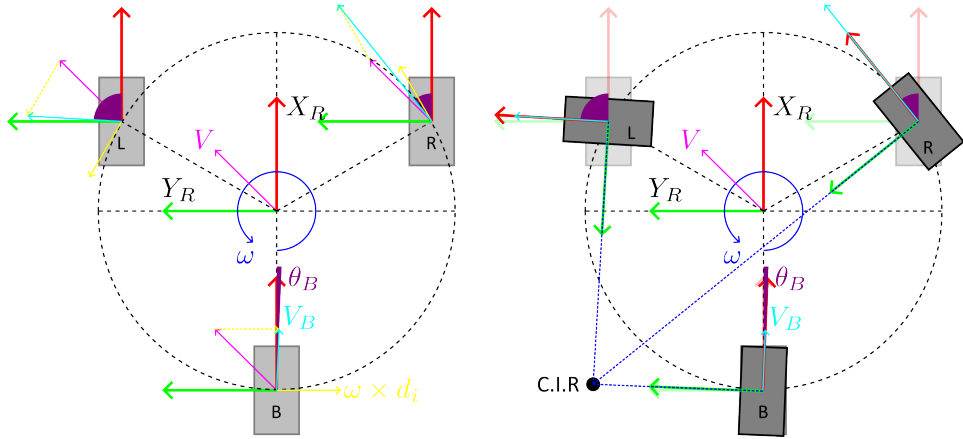


Figura 3: Cinemática inversa de un robot con configuración Y

2.2. Cinemática directa

Se denomina cinemática directa al proceso de calcular la velocidad lineal $\mathbf{V}$ y la velocidad angular $\boldsymbol{\omega}$ de la base a partir de las velocidades angulares ω_i y las orientaciones θ_i de las ruedas.

Se conocen 6 variables de entrada: las velocidades angulares ω_i y las orientaciones θ_i de cada rueda i . Se desea calcular 3 variables de salida: la velocidad lineal $\mathbf{V} = (V_x, V_y, 0)^T$ y la velocidad angular $\boldsymbol{\omega} = (0, 0, \omega_z)^T$ de la base. Por lo tanto, el sistema es sobredeterminado y se resuelve de forma robusta mediante mínimos cuadrados. En la configuración Y-invertida se usa $i \in \{F, L, R\}$ (Front, Left, Right). Para la configuración Y, se sustituye por

$i \in \{B, L, R\}$ (Back, Left, Right).

Para cada rueda i , se define el vector unitario de rodadura:

$$\mathbf{t}_i = \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \\ 0 \end{bmatrix} \quad (8)$$

La velocidad lineal de la rueda en su dirección de rodadura es:

$$\mathbf{V}_i = r\omega_i \mathbf{t}_i = r\omega_i \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \\ 0 \end{bmatrix} \quad (9)$$

Igualando (4) y (9), para cada rueda i se obtiene:

$$\begin{aligned} V_x - \omega_z d \sin \alpha_i &= r\omega_i \cos \theta_i \\ V_y + \omega_z d \cos \alpha_i &= r\omega_i \sin \theta_i \end{aligned} \quad (10)$$

En el caso ideal (sin ruido ni deslizamiento), cualquier subconjunto de 3 ecuaciones independientes permite resolver $\mathbf{V}$ y ω de forma exacta. Desde un punto de vista matemático ideal, la forma de proceder consiste en construir un sistema lineal y resolverlo mediante mínimos cuadrados.

$$A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} = \begin{bmatrix} V_x \\ V_y \\ \omega_z \end{bmatrix} \quad (11)$$

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 = (A^T A)^{-1} A^T \mathbf{b} \quad (12)$$

Como referencia, para la configuración Y-invertida ($i \in \{F, L, R\}$) se usarían las siguientes matrices A y $\mathbf{b}$:

$$A_{\text{Y-invertida}} = \begin{bmatrix} 1 & 0 & -d \sin \alpha_F \\ 0 & 1 & d \cos \alpha_F \\ 1 & 0 & -d \sin \alpha_L \\ 0 & 1 & d \cos \alpha_L \\ 1 & 0 & -d \sin \alpha_R \\ 0 & 1 & d \cos \alpha_R \end{bmatrix} \quad (13)$$

$$\mathbf{b}_{\text{Y-invertida}} = r \begin{bmatrix} \omega_F \cos \theta_F \\ \omega_F \sin \theta_F \\ \omega_L \cos \theta_L \\ \omega_L \sin \theta_L \\ \omega_R \cos \theta_R \\ \omega_R \sin \theta_R \end{bmatrix} \quad (14)$$

con $\alpha_F = 0^\circ$, $\alpha_L = \alpha$ y $\alpha_R = -\alpha$.

Para la configuración Y ($i \in \{B, L, R\}$) se usarían las siguientes matrices A y $\mathbf{b}$:

$$A_Y = \begin{bmatrix} 1 & 0 & -d \sin \alpha_B \\ 0 & 1 & d \cos \alpha_B \\ 1 & 0 & -d \sin \alpha_L \\ 0 & 1 & d \cos \alpha_L \\ 1 & 0 & -d \sin \alpha_R \\ 0 & 1 & d \cos \alpha_R \end{bmatrix} \quad (15)$$

$$\mathbf{b}_Y = r \begin{bmatrix} \omega_B \cos \theta_B \\ \omega_B \sin \theta_B \\ \omega_L \cos \theta_L \\ \omega_L \sin \theta_L \\ \omega_R \cos \theta_R \\ \omega_R \sin \theta_R \end{bmatrix} \quad (16)$$

con $\alpha_B = 180^\circ$, $\alpha_L = \alpha$ y $\alpha_R = -\alpha$.

Nótese que la solución de mínimos cuadrados dada por (12) es válida siempre que $A^\top A$ sea invertible.

En condiciones reales, las 6 ecuaciones no encajan de forma exacta por varias causas:

- ruido de sensores
- deslizamiento de ruedas
- errores de geometría o calibración
- cuantización de encoders

Aunque la expresión cerrada (12) es útil para análisis, en implementación numérica se recomienda evitar el cálculo explícito de $(A^\top A)^{-1}$, ya que por las causas mencionadas, A puede estar mal condicionada o incluso ser singular, lo que puede hacer que $A^\top A$ no sea invertible o que su inversa sea numéricamente inestable.

Desde el punto de vista de implementación, la forma recomendada es resolver mínimos cuadrados con métodos numéricamente estables, en lugar de usar la inversión explícita de $(A^\top A)$.

Para ello, pueden seguirse dos enfoques:

- Resolver directamente el problema de mínimos cuadrados mediante descomposición QR (descomposición ortogonal-triangular).

- Resolverlo mediante la pseudoinversa de Moore-Penrose:

$$\hat{\mathbf{x}} = A^\dagger \mathbf{b}$$

En este enfoque, la forma habitual de calcular $A^\dagger$ de manera robusta es la descomposición en valores singulares (SVD).

Como criterio práctico de implementación:

- **QR (descomposición ortogonal-triangular):**
 - Ventajas: menor coste computacional y mayor velocidad.
 - Inconveniente: menor robustez cuando el sistema está muy mal condicionado o casi singular.
 - Uso recomendado: estimación online de $\hat{\mathbf{x}}$ cuando el problema está razonablemente bien condicionado.
- **SVD (descomposición en valores singulares):**
 - Ventajas: mayor robustez numérica y mejor manejo de sistemas mal condicionados o casi singulares; además permite diagnosticar el rango efectivo del sistema.
 - Inconveniente: mayor coste computacional.
 - Uso recomendado: cuando se prioriza estabilidad numérica y robustez frente a ruido o mala condición.

Para decidir entre QR y SVD en este problema, puede seguirse el siguiente procedimiento:

- Construir A a partir de la geometría (d y α_i).
- Evaluar su condicionamiento una vez (por ejemplo, al arrancar), ya que A es constante si la geometría no cambia.
- Calcular el número de condición de A :

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

donde $\sigma_{\max}(A)$ y $\sigma_{\min}(A)$ son, respectivamente, el mayor y el menor valor singular de A .

- Para obtener los valores singulares, usar la descomposición en valores singulares:

$$A = U \Sigma V^T$$

donde U y V son matrices ortogonales, y Σ es una matriz diagonal (o diagonal rectangular) cuyos elementos diagonales son los valores singulares σ_i .

- Forma equivalente práctica:
 - Calcular $A^T A$.
 - Obtener sus autovalores (eigenvalues) λ_i .
 - Calcular $\sigma_i = \sqrt{\lambda_i}$.
 - Calcular explícitamente:

$$\sigma_{\text{máx}} = \sqrt{\lambda_{\text{máx}}(A^T A)}, \quad \sigma_{\text{mín}} = \sqrt{\lambda_{\text{mín}}(A^T A)}$$

- Elegir método usando umbrales orientativos de κ :
 - $\kappa < 10^3$: usar QR
 - Si $\kappa \geq 10^3$, usar SVD.
- En el código, conviene calcular el residuo como chequeo de calidad de la solución:
 - Útil para:
 - Detectar deslizamiento o lecturas anómalas.
 - Detectar que el modelo no explica bien los datos en ese instante.
 - Lanzar flags, diagnóstico o log.
 - Métrica típica:

$$r = \|A\hat{\mathbf{x}} - \mathbf{b}\|_2, \quad r_{\text{rel}} = \frac{\|A\hat{\mathbf{x}} - \mathbf{b}\|_2}{\max(\|\mathbf{b}\|_2, \varepsilon)}$$

En el ejemplo de código se toma $\varepsilon = 10^{-12}$ para evitar divisiones por cero cuando $\|\mathbf{b}\|_2$ es muy pequeña.

- Interpretación práctica:
 - $r_{\text{rel}} \leq 10^{-3}$: solución consistente.
 - $10^{-3} < r_{\text{rel}} \leq 10^{-2}$: calidad intermedia, conviene monitorizar.
 - $r_{\text{rel}} > 10^{-2}$: posible slip, ruido alto, outliers o descalibración.
- En ejecución online, actualizar únicamente $\mathbf{b}$ con las lecturas de ruedas y resolver con el método elegido.
- Repetir la evaluación solo si cambia la geometría o la calibración.

Como complemento, el siguiente ejemplo en C++/Eigen muestra una implementación directa del procedimiento anterior:

Listing 1: Receta en C++/Eigen para seleccionar QR o SVD y validar el resultado

```

using Eigen::ComputeThinU;
using Eigen::ComputeThinV;
using Eigen::JacobiSVD;
using Eigen::Matrix;
using Eigen::Matrix3d;
using Eigen::SelfAdjointEigenSolver;
using Eigen::Vector3d;

// A se construye una sola vez al arrancar el programa
Matrix<double, 6, 3> A = build_A(d, alpha);
//  $\kappa(A) = \text{sig\_max} / \text{sig\_min}$ .
Matrix3d AtA = A.transpose() * A;
SelfAdjointEigenSolver<Matrix3d> es(AtA);
// Orden ascendente
Vector3d lambda = es.eigenvalues();
double sig_min = std::sqrt(std::max(lambda(0), 0.0));
double sig_max = std::sqrt(std::max(lambda(2), 0.0));
double kappa = sig_max / std::max(sig_min, 1e-12);

...
while(data_available) {
    // En cada iteracion solo cambia b.
    // w: velocidad angular de cada rueda (3x1)
    // theta: orientacion de cada rueda (3x1)
    // r: radio de rueda
    Matrix<double, 6, 1> b = build_B(w, theta, r);
    Vector3d x_hat;

    if (kappa < 1e3)
    {
        // QR con pivoteo
        x_hat = A.colPivHouseholderQr().solve(b);
    }
    else
    {
        auto opts = ComputeThinU | ComputeThinV;
        JacobiSVD<Matrix<double, 6, 3>> svd(A, opts);
        x_hat = svd.solve(b);
    }

    // Chequeo de calidad (residuo relativo).

```

```

double res = (A * x_hat - b).norm();
double res_rel = res / std::max(b.norm(), 1e-12);
constexpr double res_rel_threshold = 1e-2;
if (res_rel > res_rel_threshold)
{
    // warning: calidad baja de la estimacion
    log_warning("High residual", res_rel);
}
}

```

A. Pseudoinversa de Moore-Penrose

Este contenido se ha retirado del cuerpo principal y se conserva en este apéndice como referencia conceptual. Aunque su valor teórico es alto, su integración directa en la narrativa principal podía introducir confusión en el flujo de lectura.

La pseudoinversa de Moore-Penrose, $A^\dagger$, generaliza la expresión de mínimos cuadrados:

- $A^\dagger$ está definida para cualquier matriz A , incluso si A no es cuadrada o no es invertible.
- La formulación de mínimos cuadrados mediante el uso de $A^\dagger$ mejora la robustez cuando las ecuaciones son casi dependientes o están afectadas por ruido:

$$\hat{\mathbf{x}} = A^\dagger \mathbf{b}$$

- Cuando A tiene rango completo en columnas (las columnas de A son linealmente independientes), la matriz $A^\top A$ es invertible y la solución de mínimos cuadrados se puede calcular mediante la fórmula cerrada $(A^\top A)^{-1} A^\top \mathbf{b}$. En este caso se cumple la equivalencia:

$$A^\dagger = (A^\top A)^{-1} A^\top$$

Desde el punto de vista numérico, para calcular $A^\dagger$ de forma robusta se utiliza la descomposición en valores singulares (SVD).